

🎓 MNNIT – Worst Thing

“Honestly, academics at MNNIT are quite demanding. Sometimes the syllabus and deadlines become stressful, especially during exams and project submissions. But that pressure also helped me learn time management and consistency.”

🎓 MNNIT – Best Things

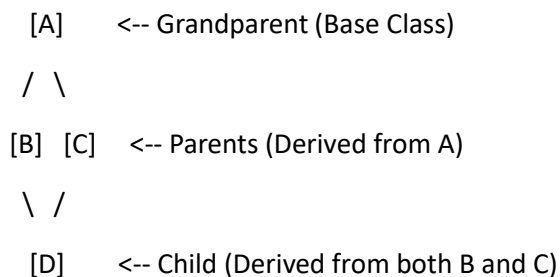
“The best thing about MNNIT is the exposure. The competitive environment, coding culture, and peer learning really push you to improve. Faculty guidance and campus life also helped me grow both technically and personally.”

❓ Why Java Does Not Support Multiple Inheritance

“Java does not support multiple inheritance using classes because it can create ambiguity problems like the **Diamond Problem**. To avoid confusion and complexity, Java allows multiple inheritance only through **interfaces**.”

The Diamond Problem is a well-known ambiguity that arises in object-oriented programming (OOP) when a language allows multiple inheritance.

It occurs when a child class inherits from two parent classes, both of which inherit from the same grandparent class. The "diamond" shape comes from the inheritance diagram:



1. Why is it a "Problem"?

The problem arises because Class D has two paths to Class A. This creates two major issues:

Method Ambiguity: If A has a method `attack()`, and both B and C override it, which version does D inherit? Does it execute B's version or C's version? The compiler or interpreter cannot decide automatically.

State Duplication: If A has a data member (e.g., `int health`), does D get two copies of `health` (one via B and one via C)? This usually leads to wasted memory and inconsistent state (e.g., updating `health` via the "B path" leaves the "C path" unchanged).

SQL – Table Creation

```
CREATE TABLE Employee (  
    employee_id INT,  
    employee_name VARCHAR(50),  
    salary INT,  
    mobile_no VARCHAR(15),  
    PRIMARY KEY (employee_id)  
);
```

First Maximum Salary

```
SELECT MAX(salary) FROM Employee;
```

Third Maximum Salary

```
SELECT MAX(salary)  
FROM Employee  
WHERE salary < (  
    SELECT MAX(salary)  
    FROM Employee  
    WHERE salary < (SELECT MAX(salary) FROM Employee)  
);
```

Fourth Maximum Salary

```
SELECT DISTINCT salary  
FROM Employee  
ORDER BY salary DESC  
LIMIT 1 OFFSET 3;
```

Composite Key Question

“Yes, a composite key can have **more than two attributes**. There is no restriction — it depends on design requirements.”

Composite Key Example

```
CREATE TABLE Employee (  
    employee_id INT,  
    employee_name VARCHAR(50),  
    salary INT,  
    mobile_no VARCHAR(15),  
    PRIMARY KEY (employee_id, mobile_no)  
);
```

Microservices in Java

“Microservices is an architecture where an application is divided into small, independent services. In Java, microservices are commonly built using **Spring Boot**, where each service runs independently and communicates through REST APIs.”

HR – Why Amdocs?

“Amdocs works on large-scale telecom and enterprise solutions, which gives exposure to real-world systems. I want to start my career in a company where I can learn industry-level development and grow technically.”

Amdocs is a **software company** that mainly works with **telecom and media companies**.

In a simple way

- It builds **software systems** for mobile networks, internet providers, and TV companies.
- Helps companies manage **customers, billing, payments, and network operations**.
- Big telecoms use Amdocs to **run their daily business smoothly**.

One-line version (for interview):

 *Amdocs provides software and services that help telecom companies manage customers, billing, and network operations efficiently.*

Can You Learn Java After Joining?

“Yes, definitely. I already know programming fundamentals and OOPS concepts. Learning Java in depth is something I’m confident about.”

Is AI Replacing Humans?

“AI will not replace humans completely. It will replace repetitive tasks, but humans are still required for decision-making, creativity, and system design.”

🌙 Can You Work in Shifts?

“Yes, I am comfortable working in shifts if required.”

📍 Any Location?

“Yes, I’m flexible with location and ready to relocate as per company requirements.”

API Handling

“I handled API calls using `async/await`. I wrapped calls inside try-catch blocks to handle errors gracefully and ensured proper response validation before updating UI or database.”

///

◆ Synchronous (Sync)

- Code runs **line by line**
- Each step **waits** for the previous one to finish
- If something is slow → **UI can freeze**

Example (sync):

```
const data = getData(); // waits here
setState(data);
```

👉 Rare in React for heavy tasks because it blocks the UI.

◆ Asynchronous (Async)

- Code **does not block** the UI
- Long tasks run in background
- Result comes later using **`async/await`** or **`.then()`**

Example (async in React):

```
useEffect(() => {
  async function fetchData() {
    const res = await fetch("/api/data");
```

```
const data = await res.json();  
setData(data);  
}  
fetchData();  
}, []);
```

👉 Most common in React (API calls, timers, events).

◆ Key Differences (easy to remember)

Sync	Async
Blocking	Non-blocking
UI may freeze	UI stays responsive
Simple tasks	API calls, DB calls
Rare in React	Very common in React

◆ Interview one-liner

Synchronous code blocks execution, while asynchronous code allows React to stay responsive by handling long tasks like API calls in the background.

Promises

“A promise represents a value that will be available in the future. It has three states: pending, fulfilled, and rejected. In my projects, async/await internally works on promises.”

Closures

“A closure is when a function remembers variables from its outer scope even after the outer function has finished execution. It helps in data hiding and maintaining state.”

Scenario: Manager Calls on Friday Evening for a Bug

“Yes, I would pick the call. If a bug affects production or users, fixing it becomes a responsibility. I believe ownership of work is important.”

Deployment & Database

“Projects were deployed on cloud platforms. DailyFit uses a NoSQL database for flexible schema and faster iteration, while attendance data is stored securely with timestamps.”

Role in Group Project

“I worked mainly on backend logic, API integration, and database design. I also collaborated with teammates to resolve integration issues.”

DSA: Pth Factor of N (Optimal)

“Iterate from 1 to $\sqrt{n}$, collect factors in order, and use symmetry to determine the pth factor in $O(\sqrt{n})$ time.”

OOPS Concepts

“OOPS includes Encapsulation, Inheritance, Polymorphism, and Abstraction.”

Encapsulation (Real-life Example)

“Capsule medicine hides internal content and exposes only what is needed. Similarly, encapsulation hides data using private variables and exposes them using methods.”

Compile-time vs Runtime Polymorphism

“Compile-time polymorphism is method overloading. Runtime polymorphism is method overriding using inheritance and dynamic binding.”

Core Java

“I am comfortable with basics like OOPS, collections, strings, exceptions, and input handling.”

SQL: GROUP BY, HAVING, ORDER BY

“GROUP BY groups records, HAVING filters groups, ORDER BY sorts the result.”

DBMS vs RDBMS

“DBMS does not support relations or constraints. RDBMS supports tables, relationships, and normalization.”

Puzzle: Man & Dog Crossing River Without Getting Wet

“The river was frozen, so the dog walked across.”

API Definition

“An API allows two software systems to communicate using defined rules, usually via HTTP methods.”

Linked List Loop Detection & Removal

“Using Floyd’s Cycle Detection Algorithm with slow and fast pointers, loop can be detected and removed in $O(1)$ space.”

Method Overloading vs Overriding

“Overloading happens in the same class with different parameters. Overriding happens in child class with same method signature.”

Hardest Time Overcome

“Balancing academics and project deadlines was challenging. I prioritized tasks, broke work into smaller parts, and stayed consistent.”

Clock Puzzle ($11 + 2 = 1$)

“On a clock, 11 plus 2 hours equals 1.”

Bridge & Bird Puzzle

“The bridge will not collapse. The bird’s weight was already supported indirectly through air pressure.”

HR – Short-term Goal

“To become a strong software engineer with good understanding of systems and clean coding.”

HR – Long-term Goal

“To grow into a technical lead who designs scalable systems.”

Why Should We Hire You

“I have strong fundamentals, project-based learning, and the ability to adapt quickly to new technologies.”

About Amdocs

Amdocs is a software company that mainly works with telecom and media companies.

In a simple way 🗣️

- It builds software systems for mobile networks, internet providers, and TV companies.
- Helps companies manage customers, billing, payments, and network operations.
- Big telecoms use Amdocs to run their daily business smoothly.

Or

Amdocs provides software and services that help telecom companies manage customers, billing, and network operations efficiently.

What You Like About Amdocs

“Large-scale systems, learning environment, and long-term career growth.”

Location & Shifts

“I am flexible with location and shifts.”

Expectation from Manager

“Clear guidance, feedback, and learning opportunities.”

Question to HR

“I would like feedback on my interview and areas where I can improve.”

Why do you want to join the company?

“I want to join this company because it works on real-world, large-scale systems. I am looking for an environment where I can apply my fundamentals, learn industry practices, and grow as an engineer. The structured work culture and learning opportunities here match my long-term goals.”

◆ PROJECT SECTION (As Asked in Interview)

Tell me about your project – Automatic Attendance System

“My Automatic Attendance System automates attendance using face recognition. It captures faces through a camera, matches them with stored data, and marks attendance with date and time. This removes manual work and reduces proxy attendance.”

What problem does it solve?

“It solves issues like proxy attendance, human error, and time consumption in manual attendance systems.”

Why did you create this project?

“I created it to understand real-world application of computer vision and automation. Attendance is a common real problem, so I wanted to build a practical solution.”

How is this project unique?

“It works offline, requires minimal user interaction, and marks attendance automatically without manual input.”

Technologies used

“Python, OpenCV, face recognition libraries, and file/database storage.”

Your role

“I designed the logic, implemented face detection and recognition, handled attendance storage, and tested accuracy.”

Team or individual?

“It was primarily an individual project.”

Why these technologies?

“Python is fast for prototyping, OpenCV is efficient for image processing, and face recognition libraries provide accurate matching.”

Technical follow-up (example answer)

“Faces are converted into encodings, compared with stored encodings using distance thresholds, and attendance is marked only once per day to avoid duplication.”

Tell me about your project – DailyFit

“.”

What problem does it solve?

Why did you create this project?

“I wanted to build a full-stack application that solves a real lifestyle problem and strengthens my backend and API skills.”

How is this project unique?

“It integrates fitness tracking and diet planning in one place with a simple user interface.”

Technologies used

“JavaScript, backend APIs, database (NoSQL), and frontend integration.”

Your role

“I handled backend logic, API integration, database design, and feature implementation.”

Team or individual?

“It was a group project, and I was responsible for backend and data handling.”

Why these technologies?

“NoSQL provides flexibility for user data, JavaScript simplifies full-stack development, and APIs help modularize features.”

Technical follow-up (example answer)

“APIs handle user requests, validate data, store it in the database, and return structured responses using async/await.”

Delete a node from a Binary Tree + Build the Tree (C++)

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
class Node {
```

```
public:
```

```
    int data;
```

```
    Node* left;
```

```
    Node* right;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        left = right = NULL;
```

```
    }
```

```
};
```

```
class BinaryTree {
```

```
public:
```

```
    Node* root;
```

```
    BinaryTree() {
```

```
        root = NULL;
```

```
    }
```

```
    void insert(int val) {
```

```
        if (!root) {
```

```
            root = new Node(val);
```

```
            return;
```

```
        }
```

```

queue<Node*> q;
q.push(root);

while (!q.empty()) {
    Node* temp = q.front(); q.pop();

    if (!temp->left) {
        temp->left = new Node(val);
        return;
    } else q.push(temp->left);

    if (!temp->right) {
        temp->right = new Node(val);
        return;
    } else q.push(temp->right);
}
}

```

```

void deleteNode(int key) {
    if (!root) return;

    queue<Node*> q;
    q.push(root);

    Node* keyNode = NULL;
    Node* temp;

    while (!q.empty()) {
        temp = q.front(); q.pop();

        if (temp->data == key)

```

```
keyNode = temp;
```

```
if (temp->left) q.push(temp->left);
```

```
if (temp->right) q.push(temp->right);
```

```
}
```

```
if (keyNode) {
```

```
    keyNode->data = temp->data;
```

```
    deleteDeepest(temp);
```

```
}
```

```
}
```

```
void deleteDeepest(Node* delNode) {
```

```
    queue<Node*> q;
```

```
    q.push(root);
```

```
    while (!q.empty()) {
```

```
        Node* temp = q.front(); q.pop();
```

```
        if (temp->left) {
```

```
            if (temp->left == delNode) {
```

```
                temp->left = NULL;
```

```
                delete delNode;
```

```
                return;
```

```
            } else q.push(temp->left);
```

```
        }
```

```
        if (temp->right) {
```

```
            if (temp->right == delNode) {
```

```
                temp->right = NULL;
```

```
                delete delNode;
```

```
        return;
    } else q.push(temp->right);
    }
}
};
```

◆ Python / Data Libraries

DataFrame

“A DataFrame is a 2-dimensional labeled data structure in Pandas, similar to a table with rows and columns. It can hold different data types and supports operations like filtering, grouping, and aggregation.”

Tensor

“A tensor is a multi-dimensional data structure.

Scalar → 0D

Vector → 1D

Matrix → 2D

Tensor → 3D or higher

Tensors are commonly used in machine learning and deep learning.”

◆ MERN / Web Development

Middleware (Express)

“Middleware is a function that runs between request and response. It is used for authentication, logging, validation, and error handling.”

Example:

```
app.use((req, res, next) => {
  console.log(req.url);
  next();
});
```

React Virtual DOM

“The Virtual DOM is a lightweight copy of the real DOM. React updates the Virtual DOM first and then efficiently updates only the changed parts in the real DOM.”

Reconciliation in React

“Reconciliation is the process where React compares the previous Virtual DOM with the new one using diffing and updates only the necessary components.”

JSON

“JSON is a lightweight data-interchange format used to send and receive data between client and server. It is easy to read and parse.”

Example:

```
{  
  "name": "Addy",  
  "age": 22  
}
```

◆ Interview Follow-up Pattern (What They Expect)

If you say:

“I used Postgres and wrote backend APIs”

They expect you to **immediately write**:

Sample API (pseudo)

GET /employees

Aggregation Function (Postgres)

SELECT department, AVG(salary)

FROM employee

GROUP BY department;

Be ready to **write**, not just explain.

◆ Routing Flow & MVC (Backend)

Routing Flow

“Client → Route → Controller → Service → Database → Response”

MVC Pattern

- **Model** → Database logic
- **View** → UI / response
- **Controller** → Handles request and business logic

“MVC separates concerns, making code scalable and maintainable.”

◆ Data Structures & Applications

Common Data Structures

- **Array** → Fast access
 - **Stack** → Undo, recursion
 - **Queue** → Scheduling
 - **Linked List** → Dynamic memory
 - **Tree** → Hierarchical data (BST)
 - **Graph** → Networks, routing
-

◆ SQL / Database

Aggregation Function

```
SELECT COUNT(*), SUM(salary), AVG(salary)
FROM employee;
```

PARTITION BY (Window Function)

```
SELECT name, salary,
RANK() OVER (PARTITION BY department ORDER BY salary DESC)
FROM employee;
```

◆ OOP Concepts

Encapsulation

“Wrapping data and methods into a single unit and restricting direct access using access modifiers.”

Polymorphism

“One method behaving differently based on context.”

- Compile time → Method overloading
 - Runtime → Method overriding
-

◆ Networking / HTTP Flow (Very Important)

“When a URL is hit:

1. Browser checks cache
 2. DNS resolves domain to IP
 3. TCP connection is established
 4. HTTP request is sent
 5. Server processes request
 6. Response is sent back
 7. Browser renders the response”
-
-

1. Tell me about yourself

“I am an MCA student with strong fundamentals in DSA, OOPS, SQL, and backend development. I have worked on real-world projects like an Automatic Attendance System and DailyFit, where I handled backend logic, APIs, and databases. I am keen to learn and work on scalable systems.”

2. Why do you want to join the company?

“I want to join because the company works on large-scale enterprise systems. It provides exposure to real production-level problems, structured learning, and long-term growth, which aligns with my career goals.”

3. Tell me about your internship

“I worked on backend development where I handled APIs, database queries, and business logic. I used modern frameworks, followed clean code practices, and collaborated with the team to deliver features.”

4. Which database did you use (Postgres)? Show table structure

“I used PostgreSQL.”

```
CREATE TABLE employee (  
    id SERIAL PRIMARY KEY,
```

```
name VARCHAR(50),  
salary INT,  
department VARCHAR(30)  
);
```

5. Which SQL aggregate functions did you use? Write syntax

"I used COUNT, SUM, AVG, MAX, MIN."

```
SELECT department, AVG(salary)  
FROM employee  
GROUP BY department;
```

7. Write a sample API you implemented

```
@app.get("/employees")  
def get_employees():  
    return {"data": "Employee List"}
```

8. Tell me about your project(s)

Automatic Attendance System

- **Problem:** Manual attendance, proxy issues
- **Uniqueness:** Face recognition, automation
- **Tech:** Python, OpenCV
- **Role:** Complete implementation
- **Type:** Individual project

DailyFit

- **Problem:** Manual fitness tracking
 - **Uniqueness:** Fitness + diet in one app
 - **Tech:** JavaScript, backend APIs, NoSQL DB
 - **Role:** Backend & database
 - **Type:** Group project
-

9. Technical follow-ups (expected)

Be ready to:

- Write API flow
- Explain DB schema
- Write SQL queries
- Explain async/await
- Explain architecture

1 Write API Flow (End-to-End)

API Flow (Spoken Explanation):

1. **Client (Frontend)** sends an HTTP request (GET/POST)
2. **Route** receives the request
3. **Middleware** runs (auth, validation, logging)
4. **Controller** handles business logic
5. **Service / DB Layer** interacts with database
6. **Response** is sent back to frontend (JSON)
7. **Frontend** displays data to user

One-line summary:

Client → Route → Middleware → Controller → Database → Response

2 Explain async / await

Answer (Simple):

async/await is used to handle **asynchronous operations** like API calls in a **synchronous-looking way**.

- async makes a function return a promise
- await pauses execution until the promise resolves

Why it is used:

- Cleaner code
- Avoids callback hell
- Better error handling using try-catch

Example (Conceptual):

```
async function getData() {  
  const res = await fetch("/api/data");  
  const data = await res.json();  
}
```

```
    return data;
}
```

One-liner for interview:

async/await helps write non-blocking code in a readable manner.

3 Explain Architecture (Backend / Project)

Typical Architecture (Amdocs-friendly):

- **Presentation Layer** – Frontend (UI)
- **Application Layer** – APIs / Controllers
- **Business Logic Layer** – Services
- **Data Layer** – Database

Flow:

UI → API → Business Logic → Database → API → UI

Why this architecture is used:

- Separation of concerns
- Easy maintenance
- Scalability
- Testability

If asked about project architecture:

“I followed a layered architecture where frontend, backend logic, and database are clearly separated.”

10. DSA: Delete node from Binary Tree (C++)

```
class Node {
public:
    int data;
    Node *left, *right;
    Node(int x) : data(x), left(NULL), right(NULL) {}
};
```

Deletion logic:

- Find target node
 - Replace with deepest node
 - Delete deepest node
-

11. DataFrame & Tensor

DataFrame:

"A 2D tabular data structure in Pandas with rows and columns."

Tensor:

"A multi-dimensional array (used in ML).

Scalar → 0D, Vector → 1D, Matrix → 2D, Tensor → 3D+"

12. MERN Questions

Middleware

"Functions that run between request and response in Express."

React Virtual DOM

"A lightweight copy of real DOM used for efficient updates."

Reconciliation

"Process of comparing old and new Virtual DOM and updating changes."

JSON

"Lightweight data format for client-server communication."

13. Data Structures & Applications

- **Stack:** Undo, recursion
 - **Queue:** Scheduling
 - **Tree:** Hierarchical data
 - **Graph:** Network routing
-

14. PARTITION clause & window functions

SELECT name, salary,

RANK() OVER (PARTITION BY department ORDER BY salary DESC)

FROM employee;

16. How hitting a URL returns a response

1. Browser checks cache
 2. DNS resolves domain to IP
 3. TCP connection
 4. HTTP request sent
 5. Server processes request
 6. Response returned
 7. Browser renders
-



AMDOCS INTERVIEW EXPERIENCE – ANSWERS

► Technical Interview

1. Tell me about yourself

"I am an MCA student with a strong interest in backend and full-stack development. I have worked on projects involving authentication, APIs, databases, and deployment. I focus on writing clean code and understanding how systems work end-to-end."

2. Which technologies did you use in your project?

"I used JavaScript, React, Node.js, Express, MongoDB, JWT authentication"

3. How did you implement security in your project?

"I implemented authentication using JWT tokens, role-based authorization, protected routes on both frontend and backend, password hashing, and HTTPS-based API communication."

4. What is JWT and how does it work in your project?

"JWT is a stateless authentication token.

After login, the server generates a token containing user data, signs it, and sends it to the client.

The client sends this token in headers for every request.

The backend verifies the token before allowing access to protected APIs."

5. What is Redux Toolkit?

"Redux Toolkit is an official library that simplifies Redux. It reduces boilerplate code and provides tools like slices, reducers, and async thunks for state management."

6. What do you understand by authentication?

“Authentication is the process of verifying the identity of a user before allowing access to resources.”

7. Why did you integrate Google Authentication?

“It provides faster login, better security, reduces password management, and improves user experience.”

8. Why use Firebase Storage along with MongoDB?

“MongoDB stores structured data like user details, while Firebase Storage is optimized for storing large files like images and documents.”

9. Difference between SQL and NoSQL & why MongoDB?

“SQL uses structured schemas and relations, while NoSQL is schema-less and scalable. MongoDB was used for flexibility and faster development.”

10. Difference between Artificial Intelligence and Generative AI

“AI focuses on decision-making and prediction, while Generative AI creates new content like text, images, or code.”

13. Tools used in CI/CD pipeline

“Git, GitHub, Jenkins/GitHub Actions, Docker, and cloud platforms.”

14. Difference between HTTP and HTTPS

“HTTPS is secure and encrypted using SSL/TLS, while HTTP is not secure.”

15. When logging in with Google, is it HTTP or HTTPS?

“It is HTTPS because authentication requires encrypted communication.”

16. For what purpose do you use Artificial Intelligence?

“I use AI for automation, recommendation systems, and improving user experience.”

17. Which websites do you use for coding?

“LeetCode, GeeksforGeeks.”

18. Explain OOPS with real-world examples

- **Encapsulation:** Capsule medicine
 - **Inheritance:** Parent–child traits
 - **Polymorphism:** Same remote, different devices
 - **Abstraction:** ATM machine
-

19. Handling different user roles in Career Crafter project

“I implemented role-based access control. Each role has specific permissions enforced at backend APIs.”

20. Which technologies do you like most?

“I like backend development, APIs, and database design.”

21. If teammates don’t work, what do you do?

“I communicate, understand issues, redistribute tasks if needed, and ensure deadlines are met.”

22. How do you motivate team members?

“By encouraging open communication, appreciating efforts, and helping when someone is stuck.”

26. Why do you want to join Amdocs?

“Amdocs works on large-scale enterprise systems and provides strong learning opportunities.”

27. Why should I hire you?

“I have strong fundamentals, project experience, and the ability to learn quickly.”

 HR INTERVIEW ANSWERS

2. If funded, which startup would you open?

"A tech platform focused on education and skill development."

3. If company supports anything, what will you do?

"I will upskill and contribute to impactful projects."

4. Expand one project with funding

"I would expand it with scalability, cloud deployment, and advanced features."

5. Team members for startup

"Skilled, motivated, and collaborative people."

6. Something not mentioned in resume

"My ability to stay calm under pressure."

7. What do you like without studying?

"Exploring tech videos and problem solving."

8. Family details

(Simple honest answer.)

9. CEO of Amdocs

"Shuky Sheffer."

10. Questions to interviewer (ask at least 3)

- What growth opportunities are there for freshers?
 - How does Amdocs support learning?
 - What makes a fresher successful here?
-

 **Amdocs Interview Experience – Exact Asked Content (Refined & Structured)**

◆ **Technical Interview**

Introduction Phase

- Interviewer first introduced himself.
 - Asked candidate to introduce himself.
 - Candidate focused mainly on **project explanation**.
 - After introduction, **cross-questioning started from projects**.
 - Interviewer deeply questioned **filter functionality** implemented in the project with technical explanation (logic, flow, implementation).
-

◆ Core Subject Questions Asked

3. System Design (Web Application)

“Suppose you have to build a website like Amazon”

Asked points:

- What **factors** will you consider?
 - Which **technologies** will you use?
 - Why those technologies?
-

4. Scalability

“How would you scale your website?”

Expected discussion:

- Load balancing
 - Horizontal scaling
 - Caching
 - Database optimization
-

5. High Traffic Handling

“If your website has a lot of requests per second, how will you handle it?”

Expected:

- Load balancer
- Caching (Redis)
- Async processing
- Database indexing

SYSTEM DESIGN – “Website like Amazon”

1 What factors will you consider?

“I will consider scalability, performance, security, availability, data consistency, and user experience. Since it’s a large-scale system, fault tolerance and monitoring are also important.”

2 Which technologies will you use?

- **Frontend:** React
 - **Backend:** Node.js with Express / Java-based services
 - **Database:** SQL for transactions, NoSQL for flexible data
 - **Cache:** Redis
 - **Search:** Elasticsearch
 - **Cloud:** AWS / Azure
 - **Messaging:** Kafka / Queue
 - **CDN:** CloudFront
-

3 Why these technologies?

- React → Fast, dynamic UI
 - Node/Java → Handles high concurrency
 - SQL → Strong transactional support
 - NoSQL → Scalability
 - Redis → Faster response
 - Cloud → Auto-scaling & reliability
-

SCALABILITY

4 How would you scale your website?

“I would scale the system horizontally by adding more servers, use load balancers to distribute traffic, cache frequent data, and optimize database queries.”

Key Techniques Explained:

✓ Load Balancing

Distributes traffic across multiple servers to avoid overload.

✓ Horizontal Scaling

Add more servers instead of increasing power of one server.

✓ Caching

Store frequently accessed data in Redis to reduce DB load.

✓ Database Optimization

- Indexing
- Read replicas
- Sharding (if needed)

HIGH TRAFFIC HANDLING

5 How do you handle high requests per second?

"I will use load balancers, caching, async processing, and database indexing to handle high traffic."

✓ Load Balancer

Distributes incoming traffic to healthy servers.

✓ Caching (Redis)

Reduces database hits for frequent queries.

✓ Async Processing

Use message queues for tasks like emails, notifications, logs.

✓ Database Indexing

Improves query performance for large datasets.

What happens if traffic spikes suddenly?

- Auto-scaling triggers
- Cache absorbs read load
- Load balancer distributes traffic
- Non-critical tasks move to async queue

1. Introduction

- Education (current + previous)
 - Achievements till now
-

Key Interview Observations (Very Important)

- **Project was the main focus**
 - Any feature mentioned (like filter/search) was **deeply cross-questioned**
 - OOPS + System Design + Scalability questions were emphasized
 - Interviewer checked:
 - Thinking ability
 - Fundamentals
 - Real-world application of concepts
 - Questions moved **from what you said next**
-

AMDOCS INTERVIEW – REAL QUESTIONS & READY ANSWERS

Interview Flow (As Experienced)

- Pehle **basics**: introduction, career goals
 - Phir **projects** (deep dive)
 - Phir **OOPS + DBMS + DSA**
 - End me **logic / feedback**
-

Difference between Function Overloading & Overriding

Overloading	Overriding
Same class	Parent–child class
Compile-time	Run-time
Different parameters	Same signature

What are Virtual Functions? One Use Case

Virtual function run-time pe decide hota hai kaunsa function call hoga.

📌 Use case:

Jab parent pointer child object ko point karta ho.

👉 Used in **runtime polymorphism**.

◆ DBMS / BACKEND QUESTIONS

Backend Application Database se kaise communicate karti hai?

- **Step-by-Step Flow (MongoDB)**
- **1 Client Request**
- Frontend se request aati hai (GET / POST).
- ---
- **2 Backend API**
- Backend (Node.js / Express) request receive karta hai.
- ---
- **3 MongoDB Driver / Mongoose**
- Backend **MongoDB driver** ya **Mongoose** use karta hai to database se connect hone ke liye.
- Example:
- `mongoose.connect("mongodb://localhost:27017/dbname");`
- ---
- **4 Query Execution**
- Backend MongoDB par query chalata hai:
- `find()`
- `insertOne()`
- `updateOne()`
- `deleteOne()`
- ---
- **5 MongoDB Response**
- MongoDB result return karta hai (JSON-like document).
- ---
- **6 Response to Client**
- Backend response ko process karke frontend ko bhej deta hai.

- **One-Line Flow (Very Important)**
 - **Frontend → Backend API → MongoDB Driver/Mongoose → MongoDB → Backend → Frontend**
 - **Why MongoDB is Easy to Communicate With?**
 - JSON-like documents
 - No complex joins
 - Native support with Node.js
 - Schema flexibility
 - **If Interviewer Asks: “Why Mongoose?”**
 - “Mongoose schema validation, middleware, and easier query handling provide karta hai, jo raw MongoDB driver se better developer experience deta hai.”
 - **One-Line Final Answer (Memorize This)**
 - “Backend MongoDB se Mongoose ke through connect karta hai, queries execute karta hai, aur result frontend ko return karta hai.”
-

◆ **PROJECT QUESTIONS (Very Deep)**

Project Explanation

- Project ka **problem statement**
- **Why you chose this topic**
- **Uniqueness**
- **Tech stack**
- **Your role**
- **Scalability**
- **GenAI integration (if asked)**

⚠ Jo feature tum bolte ho (filter, search, auth), **uska logic poochte hain.**

◆ **GEN AI / ML QUESTIONS (As per WhatsApp)**

What is GenAI?

GenAI aisa AI hota hai jo **new content generate** karta hai (text, image, code).

How will you integrate GenAI in your project?

- Chatbot
 - Resume analyzer
 - Recommendation system
 - Auto content generation
-

ML Project in Telecommunication

Examples:

- Network fault detection
 - Call drop prediction
 - Traffic optimization
-

◆ DATASET QUESTIONS

- Dataset ka **source**
 - Public / Kaggle / real data
 - Preprocessing steps
-

◆ NIT ALLAHABAD / AI-ML CURRICULUM

Soft Computing

- Neural Networks (Perceptron, MLP)
- Fuzzy Logic & Neuro-Fuzzy Systems
- Pattern recognition concepts

Data Mining

- Classification (Decision Tree, Naive Bayes, ANN, SVM)
- Clustering (K-Means, KNN)
- Association Rule Mining (Apriori)
- Regression, Model Evaluation (ROC, Bagging, Boosting)

Web Mining

- NLP basics
 - Content, structure & usage mining
 - Recommendation systems
-

◆ SCENARIO BASED QUESTIONS

Team members kaam nahi kar rahe ho to?

- Communicate
 - Understand problem
 - Redistribute tasks
 - Deadline ensure
-

◆ HR / PERSONAL

Motivation for Projects

“Real-world problem solve karna + skill improvement”

✅ AMDOCS TECHNICAL ROUND – COMPLETE PREP (FROM IMAGES)

2 Loop Detection in Linked List

Use **Floyd's Cycle Detection Algorithm**.

Logic:

- Slow pointer moves 1 step
- Fast pointer moves 2 steps
- If they meet → loop exists

Time: $O(n)$

Space: $O(1)$

3 Next Greater Element

For each element, find the next element on right that is greater.

Optimal approach:

- Use **stack**
- Traverse from right to left

Time: $O(n)$

◆ PROJECT QUESTIONS

Any one project explanation

Explain:

- Problem
 - Why this framework
 - Real-life use
 - Your role
 - Scalability
-

Constraints in SQL

- PRIMARY KEY
 - FOREIGN KEY
 - UNIQUE
 - NOT NULL
 - CHECK
-

◆ C++ CORE (VERY IMPORTANT)

Difference between C and C++

- C → Procedural
 - C++ → Object Oriented
 - C++ supports classes, inheritance, polymorphism
-

Char Pointer vs Int Pointer

- `char*` moves 1 byte

- `int*` moves 4 bytes

Pointer vs Reference

Pointer	Reference
Can be NULL	Cannot be NULL
Can change	Cannot change
Uses *	Direct

Dynamic Memory Allocation

- `new` / `delete`
- `malloc` / `free`

2D Array using Dynamic Memory

```
int** arr = new int*[r];  
for(int i=0;i<r;i++)  
    arr[i] = new int[c];
```

Delete Keyword

Used to free memory allocated using `new`.

malloc vs new

malloc	new
C style	C++
No constructor	Calls constructor
<code>void*</code>	Typed

Return type of malloc

`void*`

Can Constructor be Private?

Yes. Used in **Singleton Design Pattern**.

Friend Function

Can access private members of class.

volatile in C++

Tells compiler variable can change unexpectedly.

Used in:

- Multithreading
 - Hardware registers
-

Multilevel Inheritance & Diamond Problem

Diamond problem occurs in multiple inheritance.

Solved using **virtual base class**.

Does C also have pointers?

Yes. Pointers originated in C.

◆ SQL

Join 3 Tables

```
SELECT *
```

```
FROM A
```

```
JOIN B ON A.id=B.id
```

```
JOIN C ON B.id=C.id;
```

Sort in Descending Order

```
ORDER BY salary DESC;
```

ORDER BY vs GROUP BY

- ORDER BY → sorting
- GROUP BY → aggregation

◆ UNIX COMMANDS (ANY 5)

- ls
- cd
- pwd
- mkdir
- rm

◆ PUZZLE

Camel & 3000 bananas → optimization + logic (classic interview puzzle).

HR ROUND

Brief Intro

Education + skills + learning attitude

Family Background

Simple & honest

Relocation

“Yes, I’m comfortable.”

✅ OOPS – ANSWERS (FIRST)

1. What is OOPS?

OOPS (Object-Oriented Programming System) is a programming approach where software is designed using **objects and classes**.

It helps in **code reusability, modularity, security, and maintainability**.

2. Explain the 4 pillars of OOPS with real-life examples

(a) Encapsulation

Real-life example:

Capsule medicine — inner content is hidden, only outer shell is visible.

In code:

Private variables + public methods.

(b) Inheritance**Real-life example:**

Child inherits traits from parents.

In code:

```
class Child : public Parent {};
```

(c) Polymorphism

Same function name, different behavior.

Real-life example:

Same remote controls TV, AC, Music system.

Types:

- Compile-time → Function overloading
 - Run-time → Function overriding
-

(d) Abstraction

Showing only essential details, hiding implementation.

Real-life example:

ATM machine — user doesn't know internal process.

In code:

Using abstract class / virtual functions.

4. Types of inheritance

- Single
 - Multilevel
 - Hierarchical
 - Multiple (via interface)
 - Hybrid
-

5. What is polymorphism?

Polymorphism means **many forms** — same function behaves differently based on object or parameters.

6. How can we achieve polymorphism?

- **Compile-time:** Function overloading
 - **Run-time:** Function overriding using virtual functions
-

7. Difference between function overloading and overriding

Overloading	Overriding
Same class	Parent–child class
Different parameters	Same signature
Compile-time	Run-time

8. What are virtual functions?

A virtual function ensures that **correct function is called at runtime** based on object type, not pointer type.

9. One use case of virtual function

Used when **parent pointer points to child object** (runtime polymorphism).

11. Can constructor be private?

Yes.

Used in **Singleton Design Pattern** to restrict object creation.

12. What is a friend function?

A friend function can access **private and protected members** of a class.

13. What is diamond problem?

Occurs in **multiple inheritance** when same base class is inherited twice.

14. How is diamond problem solved in C++?

Using **virtual base class**.

15. What is abstraction vs encapsulation?

Abstraction	Encapsulation
Hides implementation	Hides data
Design level	Implementation level

DBMS – ANSWERS (SECOND)

1. What is DBMS?

DBMS is software used to **store, manage, and retrieve data** efficiently.

2. What is RDBMS?

RDBMS stores data in **tables with relations**, follows constraints and normalization.

3. Difference between DBMS and RDBMS

DBMS	RDBMS
No relations	Relations
Less secure	More secure
No normalization	Normalization

4. What are constraints in SQL?

Rules applied to columns to maintain data integrity.

5. Types of constraints

- PRIMARY KEY
 - FOREIGN KEY
 - UNIQUE
 - NOT NULL
 - CHECK
-

9. Why do we need normalization?

- Avoid duplication
 - Maintain consistency
 - Save memory
-

10. Backend application database ke sath kaise communicate karti hai?

Using **drivers/connectors** like:

Mongoose (ODM – Object Data Modeling library for MongoDB)

MongoDB Driver (official Node.js driver)

14. SQL vs NoSQL

SQL	NoSQL
Structured schema	Flexible schema
Relations	Document-based
ACID	BASE

15. Why MongoDB over SQL?

Used MongoDB because:

- Schema flexibility
 - Easy scaling
 - Faster development
-

16. What is GROUP BY?

Groups rows with same values.

17. What is HAVING?

Filters grouped data.

18. Difference between GROUP BY and ORDER BY

GROUP BY ORDER BY

Group rows Sort rows

Aggregation Sorting

19. What are aggregate functions?

Functions that operate on multiple rows.

Examples:

- COUNT
 - SUM
 - AVG
 - MAX
 - MIN
-

20. What is JOIN?

Used to combine rows from multiple tables.

21. Types of JOIN

- INNER
 - LEFT
 - RIGHT
 - FULL
-

OPERATING SYSTEM – ANSWERS (FROM ABOVE QUESTIONS ONLY)

1. What is an Operating System?

An Operating System is **system software** that acts as an interface between the user and the hardware.

It manages **CPU, memory, files, and I/O devices**.

2. What is a Process?

A process is a **program in execution**.
It has its own **memory space, program counter, and registers**.

3. What is a Thread?

A thread is the **smallest unit of execution** inside a process.
Multiple threads can exist inside a single process and share memory.

4. Difference between Process and Thread

Process	Thread
Heavy weight	Light weight
Separate memory	Shared memory
Context switch costly	Context switch cheaper

5. What is Context Switching?

Context switching is the process of **saving the state of a running process/thread and loading another one** so CPU can switch execution.

6. What is Memory Management?

Memory management is the OS function that **allocates and deallocates memory** to processes efficiently.

7. What is Dynamic Memory Allocation?

Dynamic memory allocation means allocating memory **at runtime** instead of compile time.

Examples:

- Heap memory
 - malloc, new
-

8. What is Heap and Stack?

Stack	Heap
Automatic memory	Manual memory
Function calls	Dynamic allocation

Stack	Heap
Faster	Slower

9. What is Deadlock?

Deadlock is a situation where **two or more processes wait indefinitely** for resources held by each other.

10. What is Race Condition?

A race condition occurs when **multiple processes access shared data simultaneously**, and output depends on execution order.

11. What is Multitasking?

Multitasking allows the OS to **run multiple processes seemingly at the same time** by rapid context switching.

12. What is Scheduling?

Scheduling is the OS mechanism that **decides which process gets CPU time**.

13. What is Paging? *(Indirectly related to memory management asked)*

Paging is a memory management technique where memory is divided into **fixed-size pages**.

14. Why do we need OS?

- Efficient resource usage
 - Process management
 - Memory management
 - Security
-

CN (Computer Networks) – ANSWERS

1. Difference between HTTP and HTTPS

HTTP	HTTPS
Not secure	Secure
Data in plain text	Data encrypted
No SSL/TLS	Uses SSL/TLS
Port 80	Port 443

2. Which is more secure – HTTP or HTTPS?

HTTPS is more secure because it encrypts data using **SSL/TLS**, preventing data theft and tampering.

3. What happens when you hit a URL in a browser?

1. Browser checks cache
 2. DNS resolves domain to IP
 3. TCP connection is established
 4. HTTP/HTTPS request sent
 5. Server processes request
 6. Response returned
 7. Browser renders content
-

4. Explain DNS to IP resolution

DNS converts a **domain name** (google.com) into an **IP address** so the browser can locate the server.

5. Explain client–server request–response flow

Client sends request → Server processes it → Server sends response → Client displays result.

6. What protocol is used when logging in with Google?

HTTPS is used to ensure encrypted and secure authentication.

7. What is HTTPS and how it provides security?

HTTPS uses **SSL/TLS encryption**, ensuring:

- Data confidentiality

- Data integrity
 - Server authentication
-

8. What is API?

API (Application Programming Interface) allows **two software systems to communicate** using defined rules.

9. How does frontend communicate with backend?

Frontend sends **HTTP/HTTPS requests** (GET, POST, PUT, DELETE) to backend APIs and receives responses (usually JSON).

✅ SQL – ANSWERS (Heavily Asked)

1. Difference between SQL and NoSQL

SQL	NoSQL
Structured schema	Flexible schema
Tables	Documents
ACID	BASE
MySQL	MongoDB

2. Why did you use MongoDB instead of SQL?

MongoDB is used for:

- Flexible schema
 - Faster development
 - Easy scalability
 - JSON-like data storage
-

7. What are aggregate functions?

Functions that operate on **multiple rows** and return a single value.

8. Syntax of aggregate functions

COUNT(column)

SUM(column)

AVG(column)

MAX(column)

MIN(column)

12. What is PARTITION BY?

Used in **window functions** to divide data into partitions.

13. Write syntax of window function

SELECT name, salary,

RANK() OVER (PARTITION BY dept ORDER BY salary DESC)

FROM emp;

14. Difference between ORDER BY and GROUP BY

- **ORDER BY** → sorts rows
 - **GROUP BY** → groups rows for aggregation
-

DSA – ANSWERS (From Notes + Images)

1. What is a Linked List?

A linked list is a **linear data structure** where elements (nodes) are stored non-contiguously and each node points to the next node using a pointer.

5. Time Complexity of Algorithms

Time complexity tells **how execution time grows with input size**.

Common:

- $O(1)$ – Constant
- $O(\log n)$ – Logarithmic
- $O(n)$ – Linear
- $O(n^2)$ – Quadratic

Big O worst

Omega best

Theta Average

6. Difference between Array and Linked List

Array	Linked List
Contiguous memory	Non-contiguous
Fixed size	Dynamic size
Fast access	Slow access

7. Tree Data Structure

A tree is a **hierarchical data structure** with a root node and child nodes.

8. What is BST?

Binary Search Tree is a binary tree where:

- Left child < Root
 - Right child > Root
-

9. Tree Implementation (Concept)

Each node has:

- Data
- Left pointer
- Right pointer

Used for hierarchical storage.

10. Sliding Window Technique

Used to optimize problems involving **subarrays or substrings** by maintaining a window.

Time: $O(n)$

11. HashMap – Working & Use Case

Stores data in **key-value pairs** using hashing.

Use cases:

- Frequency counting
- Fast lookup

Time: $O(1)$ average

13. Fibonacci using Recursion

```
int fib(int n){  
    if(n<=1) return n;  
    return fib(n-1)+fib(n-2);  
}
```

14. Reverse using Recursion

```
void rev(string &s,int l,int r){  
    if(l>=r) return;  
    swap(s[l],s[r]);  
    rev(s,l+1,r-1);  
}
```

15. Selection Sort

Repeatedly selects **minimum element** and places it at correct position.

Time: $O(n^2)$

16. Difference between Stack and Queue

Stack	Queue
-------	-------

LIFO	FIFO
------	------

push/pop	enqueue/dequeue
----------	-----------------

 **C / C++ CORE – ANSWERS**

1. Difference between C and C++

C	C++
Procedural	Object Oriented
No classes	Classes & objects
No OOPS	Supports OOPS

2. What are Pointers?

Pointers store the **address of a variable**.

3. Pointer Declaration & Working

```
int x = 10;  
int* p = &x;  
p stores address of x.
```

4. Pointer vs Reference

Pointer	Reference
Can be NULL	Cannot be NULL
Can change	Cannot change
Uses *	Direct

5. Char Pointer vs Int Pointer

- char* increments by **1 byte**
 - int* increments by **4 bytes**
-

6. Dynamic Memory Allocation

Memory allocated **at runtime** using:

- new / delete
 - malloc / free
-

7. malloc vs new

malloc **new**

C style C++

No constructor Calls constructor

Returns void* Typed

8. Return Type of malloc

void*

9. delete Keyword

Used to free memory allocated using new.

11. What is volatile keyword?

Tells compiler that variable **can change unexpectedly**.

PUZZLES – ANSWERS

1 Camel and 3000 Bananas Puzzle

Problem (short):

A camel must carry 3000 bananas across a desert. The camel can carry max 1000 bananas at a time and eats 1 banana per km.

1st phase 3000 to 2000, km travelled 200km 3 right hand side , 2 return

2nd phase 2000 to 1000, km travelled 333.33 km 2 right hand , 1 return

3rd phase 1000 banana , km travelled 466.67 km 1 right only and destination reached

Max banana = 533

2 Logical Reasoning Puzzle (Generic)

Example asked earlier:

11 + 2 = 1 — How?

Answer:

“On a clock, 11 plus 2 hours becomes 1.”

HR / BEHAVIORAL – ANSWERS

2 Career Goals

Short-term:

“To become a strong software engineer with good problem-solving skills.”

Long-term:

“To grow into a senior engineer or technical lead working on scalable systems.”

3 Why do you want to join Amdocs?

“Amdocs works on large-scale enterprise and telecom systems. It provides exposure to real production problems and strong learning opportunities, which aligns with my career goals.”

4 Why should we hire you?

“I have strong fundamentals, project-based learning, and a positive learning attitude. I adapt quickly and take ownership of my work.”

9 How do you handle team members who don't work?

“I first communicate and understand the issue. If needed, I help them or redistribute tasks to ensure deadlines are met.”

10 How do you motivate your team?

“By appreciating efforts, helping when someone is stuck, and keeping communication open.”

✓ HR-RELATED ANSWERS (AMDOCS)

4. Is there anything not mentioned in your resume?

“Yes, my ability to stay calm under pressure and keep learning continuously is not fully reflected in my resume.”

7. Where do you see yourself in the future?

“I see myself contributing to complex projects and taking more responsibilities in system design.”

◆ Company & Motivation

8. Why do you want to join Amdocs?

“Amdocs works on large-scale enterprise and telecom systems. It provides strong learning opportunities and exposure to real-world problems.”

9. What do you know about Amdocs?

"Amdocs is a global company providing software and services to telecom and enterprise clients."

10. What did you find good about the company?

"The learning culture, global exposure, and long-term career growth opportunities."

11. Why should we hire you?

"I have strong fundamentals, project experience, and a positive learning attitude. I take ownership of my work."

14. Where do you want to work and in which shifts?

"I am open to any location and shift as per company requirements."

◆ **Behavioral / Situational**

15. If your manager calls on Friday evening due to a bug, would you pick the call?

"Yes, I would pick the call. Fixing production issues is part of responsibility."

16. If team members are not working, what will you do?

"I will communicate, understand their issues, and help redistribute tasks to meet deadlines."

17. How do you motivate your team members?

"By appreciating efforts, supporting them when stuck, and maintaining clear communication."

18. Hardest time you faced and how you handled it

"Balancing academics and project deadlines was challenging. I managed it by planning tasks and staying consistent."

19. How do you handle pressure?

"I stay calm, break problems into smaller parts, and focus on solutions."

◆ **Learning & Adaptability**

20. Can you learn new technologies after joining?

“Yes, I am confident in learning new technologies quickly.”

21. Are you okay working on any technology?

“Yes, I am open to working on any technology assigned.”

◆ **AI / Future-oriented**

22. Do you think AI is replacing humans?

“No. AI will automate repetitive tasks, but humans are needed for decision-making and creativity.”

23. For what purpose do you use AI?

“For automation, improving productivity, and solving real-world problems.”

◆ **Certifications & Activities**

24. Explain IBM SkillsBuild certification

“It focuses on industry-relevant skills like cloud, AI basics, and professional development.”

25. What did you learn from your AI certificate?

“Basics of AI concepts and how they are applied in real-world scenarios.”

26. Where do you apply this AI knowledge?

“In automation tasks and improving project features.”

27. What did you learn from hackathons?

“Teamwork, time management, and building solutions under time pressure.”

◆ **Personal Interests**

28. Which websites do you visit for coding?

“LeetCode, GeeksforGeeks, Stack Overflow, and documentation.”

29. What do you like apart from studies?

“Exploring technology and solving logical problems.”

◆ HR Closing

30. Do you have any questions for us? (Ask at least 3)

- What learning opportunities are provided to freshers?
 - How does Amdocs support career growth?
 - What qualities make someone successful at Amdocs?
-

✚ PUZZLES – ANSWERS (AMDOCS)

1 Man–Dog–River Puzzle

Question:

One side of a river a man is standing, on the other side a dog. The dog crossed the river without getting wet. How?

Answer:

“The river was frozen, so the dog walked across.”

2 $11 + 2 = 1$ (Clock Puzzle)

Question:

$11 + 2 = 1$ — how is this possible?

Answer:

“On a clock, 11 plus 2 hours becomes 1.”

3 Bridge, Car & Bird Puzzle

Question:

A bridge has capacity 1000 kg. A 1000 kg car is on the bridge. A bird sits on the car. Will the bridge collapse?

Answer:

“No, the bridge will not collapse.

The bird’s weight was already supported by air pressure before landing, so there is no sudden extra load on the bridge.”

(Some interviewers also accept: the bird’s weight is negligible.)

5 Magic Square Puzzle

Question (general):

What is a magic square?

Answer:

"A magic square is a square matrix where the sum of numbers in each row, column, and both diagonals is the same."

(If asked to solve, explain approach step-by-step.)

 SYSTEM DESIGN – ANSWERS

1. How would you design a website like Amazon?

"I would design it using a **microservices architecture**.

Frontend for UI, backend services for users, orders, payments, search, and inventory.

Use load balancer, caching, database sharding, and cloud deployment."

2. What components are required for a large-scale web application?

- Frontend (React)
 - Backend APIs
 - Database
 - Load balancer
 - Cache
 - Authentication
 - Monitoring & logging
-

3. How would you design a scalable backend system?

"By using stateless APIs, horizontal scaling, caching, database indexing, and asynchronous processing."

4. How do you handle millions of users at the same time?

- Load balancer
 - Multiple backend servers
 - Caching
 - Database replication
 - Auto-scaling
-

5. What factors do you consider while designing a system?

- Scalability

- Security
 - Performance
 - Availability
 - Maintainability
-

6. How would you design an authentication system?

“Using JWT-based authentication with secure password hashing and role-based access control.”

7. How would you design a search feature for large data?

- Indexed database
 - Pagination
 - Caching
 - Search engines (like Elasticsearch)
-

✅ SCALABILITY – ANSWERS

1. What is scalability?

Scalability is the ability of a system to **handle increased load** without performance degradation.

2. Vertical vs Horizontal Scaling

Vertical	Horizontal
Increase server power	Add more servers
Limited	Highly scalable

3. How would you scale your existing project?

- Add caching
 - Use load balancer
 - Optimize DB queries
 - Horizontal scaling
-

4. What happens when traffic suddenly increases?

“If not scaled, servers slow down or crash. Auto-scaling and load balancing handle this.”

5. How do you ensure high availability?

- Multiple servers
 - Failover
 - Load balancing
 - Monitoring
-

HIGH TRAFFIC HANDLING – ANSWERS

1. How do you handle high traffic?

- Load balancer
 - Caching
 - CDN
 - Asynchronous jobs
-

2. What is a load balancer?

A load balancer distributes traffic across multiple servers to avoid overload.

3. How does load balancing improve performance?

It prevents a single server from handling all requests.

4. What happens if one server goes down?

Traffic is redirected to healthy servers.

5. How do you prevent server overload?

- Rate limiting
 - Caching
 - Auto-scaling
-

CACHING / REDIS – ANSWERS

1. What is caching?

Caching stores frequently used data in fast memory.

2. Why is caching used?

To reduce database load and improve response time.

3. What data should be cached?

- Frequently accessed data
 - Read-heavy data
 - Non-critical real-time data
-

4. How does Redis improve performance?

Redis stores data in memory, making access extremely fast.

5. Where would you use Redis in your project?

- Session storage
 - Authentication tokens
 - Frequently fetched data
-

CI / CD – ANSWERS

1. What is CI/CD?

CI/CD automates **build, test, and deployment**.

2. Stages of CI/CD pipeline

- Code commit
 - Build
 - Test
 - Deploy
-

3. Tools used in CI/CD

GitHub, Jenkins, GitHub Actions, Docker.

4. How does CI/CD help?

Faster deployment, fewer bugs, automation.

5. What happens if build fails?

Pipeline stops and developer fixes the issue.

✓ AI vs GEN AI – ANSWERS

1. What is Artificial Intelligence?

AI enables machines to **make decisions and predictions**.

2. What is Generative AI?

GenAI creates **new content** like text, images, or code.

3. Difference between AI and GenAI

- AI → decision & prediction
 - GenAI → content generation
-

4. Where is AI used?

Fraud detection, recommendation systems, automation.

5. Where can GenAI be used?

Chatbots, resume screening, content generation.

✓ ML IN TELECOM – ANSWERS

1. How is ML used in telecom?

- Call drop prediction
 - Network optimization
 - Fault detection
-

2. Problems ML solves in telecom

- Traffic congestion

- Network failures
 - Customer churn
-

3. Call drop prediction

ML analyzes signal strength, traffic, and location.

4. Network performance improvement

Predicts failures and optimizes routing.

DATASET / DATA HANDLING – ANSWERS

1. What is a dataset?

A dataset is a **collection of data** used for analysis or training models.

2. Dataset source

“Kaggle / public datasets / project-generated data.”

3. Preprocessing steps

- Cleaning
 - Normalization
 - Removing duplicates
-

4. Handling missing data

- Remove rows
 - Fill with mean/median
-

5. Train-test split

“Usually 70–30 or 80–20.”

RESUME-BASED CROSS QUESTIONING – ANSWERS

Why did you mention this technology?

“Because I used it practically in my project.”

Can you explain this feature?

Explain **logic + flow + use case**.

Can you write the code/query?

Write **basic working logic**, not full app.

Challenges faced

- Integration issues
- Performance optimization
- Learning curve

The biggest challenge was arranging Raspberry Pi components during the Mahakumbh period in Prayagraj, when online deliveries were delayed. Because of this, I personally went to local electronics markets to search for compatible components like power supply, camera module, and connectors.

It took time and effort to verify compatibility and quality, but this helped me understand the hardware better. Finally, I assembled everything successfully and completed the project on time.

CODING WEBSITES / LEARNING SOURCES

Coding practice websites

LeetCode, GeeksforGeeks, Stack Overflow.

Why LeetCode / GFG?

LeetCode → DSA practice

GFG → Concept clarity

How do you use documentation?

“I read official docs for syntax and best practices.”

TOP 15 ML INTERVIEW – ANSWERS

1. What is Machine Learning?

Machine Learning is a subset of AI where machines **learn from data and improve performance without being explicitly programmed.**

2. Difference between Artificial Intelligence and Machine Learning

Artificial Intelligence	Machine Learning
Broad concept	Subset of AI
Mimics human intelligence	Learns from data
Rule-based + learning	Data-driven

3. What are the types of Machine Learning?

- **Supervised Learning**
 - **Unsupervised Learning**
 - **Reinforcement Learning**
-

4. What is supervised learning?

Learning using **labeled data**, where input and correct output are known.

Example: Spam detection.

5. What is unsupervised learning?

Learning using **unlabeled data** to find patterns.

Example: Customer segmentation.

6. What is a dataset?

A dataset is a **collection of data** used to train and test an ML model.

7. What are features and labels?

- **Features** → Input variables
 - **Labels** → Output / target variable
-

8. What is classification and regression?

- **Classification** → Output is a category (spam / not spam)

- **Regression** → Output is a number (price prediction)
-

9. What is overfitting?

When a model learns training data too well but performs poorly on new data.

10. What is underfitting?

When a model is too simple and fails to capture patterns in data.

11. What is accuracy?

Accuracy measures **how many predictions are correct** out of total predictions.

12. What is training data and testing data?

- **Training data** → Used to train the model
 - **Testing data** → Used to evaluate performance
-

13. What is data preprocessing?

Data preprocessing is cleaning and preparing data before training.

Includes:

- Handling missing values
 - Normalization
 - Removing duplicates
-

14. What is an ML model?

An ML model is a **mathematical representation** learned from data to make predictions.

15. Where is Machine Learning used in real life?

- Recommendation systems
- Fraud detection
- Spam filtering
- Healthcare
- Telecom (call drop prediction)

✓ TOP 15 ARTIFICIAL INTELLIGENCE (AI) – ANSWERS

1. What is Artificial Intelligence?

Artificial Intelligence is the ability of machines to **simulate human intelligence** like thinking, learning, and decision-making.

2. What are the different types of AI?

- **Narrow AI**
 - **General AI**
 - **Super AI** (theoretical)
-

3. What is Narrow AI?

Narrow AI is designed to perform **one specific task**.

Example: Google Assistant, recommendation systems.

4. What is General AI?

General AI can perform **any intellectual task that a human can do**.
(It does not exist yet.)

5. Difference between AI and Machine Learning

AI	ML
Broad concept	Subset of AI
Mimics intelligence	Learns from data
Can be rule-based	Data-driven

6. What is Machine Learning and its relation to AI?

Machine Learning is a part of AI where machines **learn patterns from data** instead of being explicitly programmed.

7. What is Deep Learning?

Deep Learning is a subset of ML that uses **neural networks with multiple layers** to learn complex patterns.

8. What is a Neural Network?

A neural network is a model inspired by the **human brain**, made of neurons that process data in layers.

9. What is supervised learning?

Learning using **labeled data** where input and output are known.

Example: Email spam detection.

10. What is unsupervised learning?

Learning from **unlabeled data** to find hidden patterns.

Example: Customer clustering.

11. What is reinforcement learning?

Learning by **trial and error**, using rewards and penalties.

Example: Game playing AI.

12. Where is AI used in real life?

- Healthcare
 - Finance
 - Telecom
 - Recommendation systems
 - Automation
-

13. Advantages of AI

- Faster processing
- Accuracy
- Automation

- 24x7 availability
-

14. Limitations of AI

- High cost
 - Requires quality data
 - No emotions or creativity
 - Ethical risks
-

15. Ethical issues in AI

- Bias
 - Privacy issues
 - Job displacement
 - Misuse of data
-

TOP 15 GENERATIVE AI (GenAI) – ANSWERS

1. What is Generative AI?

Generative AI creates **new content** like text, images, audio, or code.

2. Difference between Generative AI and traditional AI

- Traditional AI → predicts or classifies
 - Generative AI → creates new data/content
-

3. What are Large Language Models (LLMs)?

LLMs are AI models trained on large text data to **understand and generate human-like language**.

Example: GPT.

4. How does Generative AI generate text?

By predicting the **next word/token** based on previous context using probability.

5. How does Generative AI generate images?

Using models like **diffusion models** that gradually create images from noise.

6. What is a Transformer model?

A transformer is a neural network architecture that uses **attention mechanism** to understand context.

7. What is prompt engineering?

Designing effective input prompts to get **better and accurate output** from GenAI models.

8. What is fine-tuning?

Training a pre-trained model on **custom data** to improve performance for a specific task.

9. Common applications of Generative AI

- Chatbots
 - Content generation
 - Code generation
 - Image creation
-

10. What is ChatGPT?

ChatGPT is a conversational AI model based on **Large Language Models** that generates human-like responses.

11. What are diffusion models?

Models used mainly for **image generation**, where noise is gradually removed to form images.

12. Limitations of Generative AI

- Hallucinations
 - Data bias
 - High compute cost
 - Lack of true understanding
-

13. Ethical concerns in Generative AI

- Fake content

- Plagiarism
 - Privacy violations
 - Misinformation
-

14. How can Generative AI be integrated into projects?

- Chatbots
 - Resume screening
 - Recommendation engines
 - Content automation
-

15. Difference between ML models and GenAI models

ML Models	GenAI Models
Predict outcomes	Generate content
Classification/regression	Text/image/code generation
Smaller models	Large models

◆ PART 1

INTRO / PROFILE + MCA JOURNEY (Resume-Driven)

1. Walk me through your resume.

“Currently, I am pursuing MCA from MNNIT Allahabad. I completed my BSc earlier, and during MCA I focused heavily on practical learning through projects and problem solving. I have built projects like an Automatic Attendance System using ML and IoT, and a full-stack MERN application called DailyFit. Alongside this, I regularly practice DSA and have solved more than 940 problems on coding platforms.”

2. Tell me about your MCA journey so far.

“My MCA journey has been very learning-oriented. Initially, I focused on strengthening my fundamentals. Gradually, I shifted towards applying concepts in real projects like ML-based systems and full-stack applications. I also participated in contests and technical activities, which improved my problem-solving and teamwork skills.”

3. Why did you choose MCA after BSc?

“After BSc, I realized I wanted deeper technical exposure and industry-oriented skills. MCA provided structured learning in software development, system design, and practical projects, which aligned better with my career goals.”

4. How do your projects align with industry requirements?

“My projects focus on real-world problems like automation, scalability, and user management. The attendance system deals with accuracy, privacy, and deployment constraints, while DailyFit focuses on user experience, booking logic, and scalable backend design — all of which are common industry challenges.”

◆ PART 1 (Continued)

PROJECT: AUTOMATIC ATTENDANCE SYSTEM (High-Depth, Resume-Driven)

5. Explain your Automatic Attendance System end-to-end.

“The system captures a person’s face using a camera connected to Raspberry Pi, detects and recognizes the face using ML models, and marks attendance automatically. Attendance data is stored locally, announced via voice output, and shared securely. The entire system works offline to ensure privacy.”

6. Why did you choose face recognition instead of fingerprint or RFID?

“Face recognition is contactless and faster. Unlike fingerprint or RFID, it avoids hardware dependency on users and reduces proxy attendance, making it more suitable for classrooms.”

7. Why did you use Raspberry Pi in this project?

“Raspberry Pi provides a compact, low-cost platform capable of running ML models at the edge. It allowed me to deploy the system as a standalone device without relying on external servers.”

8. How does face detection differ from face recognition in your system?

“Face detection identifies whether a face is present in the frame, while face recognition identifies **whose face it is** by comparing extracted features with stored data.”

9. What is a 128-D facial feature vector?

“It is a numerical representation of a face generated by the recognition model. Each face is converted into a 128-dimensional vector, which makes comparison efficient and accurate.”

10. How did you achieve 95% accuracy?

"I achieved this by using high-quality facial encodings, capturing images under multiple angles, handling lighting variations, and tuning threshold values for matching."

11. How does lighting variation affect recognition and how did you handle it?

"Poor lighting can distort facial features. I handled this by preprocessing images, normalizing inputs, and capturing multiple samples per user."

12. Why did you use dlib along with OpenCV?

"OpenCV is used mainly for image processing and face detection, while dlib provides robust face recognition and feature extraction. Using both gave better accuracy."

13. How does NumPy help in this project?

"NumPy is used for efficient numerical operations on facial feature vectors, making comparison and distance calculations faster."

14. Why did you keep the system offline?

"To ensure data privacy, reduce dependency on internet connectivity, and make the system reliable in environments with limited network access."

15. How did you ensure data privacy?

"All facial data and attendance records are stored locally on the device. No data is sent to external servers."

16. How does the voice announcement feature work?

"Once attendance is marked, the system triggers a text-to-speech module that announces the person's name."

17. Why did you use pyttsx3 instead of cloud TTS?

"pyttsx3 works offline, which matched the privacy and offline design requirement of the project."

18. How is attendance data stored and managed?

“Attendance is stored locally in structured files with date and time stamps, making retrieval and sharing easy.”

19. Why did you implement automated image deletion?

“To avoid repeated recognition conflicts and manage limited storage on Raspberry Pi.”

20. What happens if the same person comes twice?

“The system checks if attendance for that person is already marked for the session and avoids duplication.”

21. How would you scale this system for 200 students?

“I would optimize face matching using indexing, improve camera placement, and possibly batch recognition for smoother processing.”

22. What challenges did you face while deploying on Raspberry Pi?

“Limited memory, processing constraints, and performance optimization were key challenges.”

◆ PART 2

PROJECT: DAILYFIT (MERN STACK)

(Resume-driven, Amdocs interview style)

Aditya_Maurya_Resume

1. Explain DailyFit project end-to-end.

“DailyFit is a MERN stack web application that connects gym users with nearby gyms offering short-term access. Users can search gyms, select date and time slots, and book 1-day passes. Gym owners manage their gyms through a dashboard where they can update details, prices, and track bookings.”

2. What real-world problem does DailyFit solve?

“Many people don’t want long-term gym memberships. DailyFit solves this by allowing flexible, short-term gym access while helping gyms monetize unused capacity.”

3. Why did you choose MERN stack?

“MERN allows fast development using a single language (JavaScript) across frontend and backend, and it scales well for web applications.”

4. Why React for frontend?

“React provides component-based architecture, efficient UI updates, and better user experience for dynamic pages like booking flows and dashboards.”

5. Why Node.js + Express for backend?

“Node.js handles multiple requests efficiently, and Express simplifies routing, middleware handling, and API development.”

6. Why MongoDB instead of a relational database?

“MongoDB offers schema flexibility, which is useful when user and gym data evolve. It also integrates naturally with JSON-based APIs.”

7. How does booking management work internally?

“When a user selects a gym, date, and time slot, the backend checks availability, creates a booking entry, and updates the slot status to prevent conflicts.”

8. How do you handle date and time slot availability?

“Slots are stored with availability flags. Before confirming a booking, the backend verifies whether the selected slot is still available.”

9. Explain the gym owner dashboard flow.

“Gym owners log in to the dashboard, manage gym details, view bookings, check in users, and update prices or timings.”

10. How do you differentiate gym users and gym owners?

“Each account has a role field. Based on this role, access is granted to either user features or owner-specific dashboards.”

11. How do you prevent double booking?

“Before inserting a new booking, the backend checks if a booking already exists for that gym, date, and time slot.”

12. How do users find nearby gyms?

“Users search gyms based on location data stored in the database and filtered on the backend.”

13. How would you handle high traffic during booking time?

“I would use caching for frequently accessed data, optimize database queries, and scale backend services horizontally.”

14. What security measures did you implement?

“I implemented authentication, role-based access control, protected APIs, and validation checks on the backend.”

15. If payment fails, how is consistency maintained?

“If payment fails, the booking is not confirmed and the slot remains available, ensuring data consistency.”

16. What improvements would you add to DailyFit?

“I would add real-time notifications, advanced search filters, analytics for gym owners, and better scalability using caching.”

◆ PART 3**TECH STACK & TOOLS + DSA & PROBLEM SOLVING**

(Strictly from your resume, non-core subjects only)

◆ PROGRAMMING / LIBRARIES**1. Where have you used Python in real projects?**

“I used Python extensively in my Automatic Attendance System for face recognition logic, image processing, numerical computation, and system integration on Raspberry Pi.”

2. Why is Python suitable for ML projects?

“Python has strong ML libraries, simple syntax, fast prototyping ability, and good community support, which makes it ideal for ML-based projects.”

3. Where did you use C++ in problem solving?

"I use C++ for competitive programming and DSA problems because of its speed, STL support, and memory control."

4. Which data structures do you use most in competitive coding?

"I mostly use arrays, vectors, hash maps, stacks, queues, and trees depending on the problem."

◆ FRAMEWORKS / LIBRARIES

5. Why did you use Tkinter?

"Tkinter was used to build a simple GUI for interacting with the attendance system, making it user-friendly without heavy overhead."

6. Why Tailwind CSS?

"Tailwind CSS allows rapid UI development using utility classes and keeps the design consistent without writing large CSS files."

7. How is Express different from plain Node.js?

"Node.js provides the runtime, while Express is a framework that simplifies routing, middleware, and API creation."

◆ TOOLS

8. How do you use Git in team projects?

"I use Git for version control, feature branching, committing changes, and resolving conflicts while collaborating."

9. What is your GitHub workflow?

"I follow a simple workflow: create feature branches, commit logically, push changes, and merge after testing."

10. How do you test APIs using Thunder Client?

"I send requests with different payloads, test response codes, validate responses, and check error handling."

11. What challenges did you face using Raspberry Pi?

“Limited processing power, memory constraints, and performance optimization were the main challenges.”

◆ DSA & PROBLEM SOLVING (NON-CORE THEORY)

12. You solved 940+ problems — how do you approach a new problem?

“I first understand the problem, identify constraints, think of a brute-force approach, and then optimize it.”

13. Which type of problems do you find most challenging?

“Graph and advanced DP problems are challenging because they require deeper logical thinking.”

14. How do you optimize a brute-force solution?

“By using better data structures, reducing repeated computations, and applying techniques like hashing or sliding window.”

15. How do contests help you improve?

“Contests improve my speed, accuracy, and ability to think under time pressure.”

16. What did you learn from LeetCode Weekly Contests?

“I learned time management, quick problem analysis, and handling tricky edge cases.”

17. How do you handle time pressure in contests?

“I prioritize easier problems first and keep track of time to maximize score.”

◆ PART 4

ACHIEVEMENTS, ACTIVITIES, LEADERSHIP & BEHAVIORAL (NON-HR THEORY)

◆ ACHIEVEMENTS & ACTIVITIES

1. What did you learn from Application Club?

“Being part of the Application Club helped me understand how coding contests are designed, how problem difficulty is balanced, and how edge cases are framed. It also improved my logical thinking.”

2. What kind of questions did you test in contests?

"I tested DSA-based problems including arrays, strings, hashing, and logical problems, focusing on correctness and edge cases."

3. You secured 4th position in OPC 2.0 — what does that say about you?

"It reflects my problem-solving ability, consistency in practice, and ability to perform under competitive conditions."

4. How does being part of the cricket team help professionally?

"Cricket taught me teamwork, discipline, leadership, and handling pressure — all of which are important in professional environments."

5. What leadership experience did you gain as Event Handler Head?

"I handled coordination, task distribution, and on-ground problem solving. It improved my communication and decision-making skills."

◆ BEHAVIORAL / SITUATIONAL (NON-HR THEORY)**6. If a bug is found in your deployed project, how do you debug it?**

"I first reproduce the issue, check logs, isolate the faulty module, fix the root cause, test thoroughly, and then redeploy."

7. How do you handle failure in a contest or project?

"I analyze what went wrong, identify gaps in my approach, and work on improving those areas."

8. How do you prioritize tasks during exams and projects?

"I plan tasks in advance, set priorities based on deadlines, and break work into smaller achievable parts."

9. How do you learn a new technology quickly?

"I start with basics, read official documentation, build a small project, and then gradually deepen my understanding."

NEXT PART (FINAL PART – PART 5)

Reply **next** and I'll finish with:

◆ PART 5

- **Future & Company-Fit**
- **Which project you're most proud of**
- **6-month product vision**
- **How your skills fit Amdocs**
- **Enterprise systems readiness**

This will complete the **full Amdocs resume-driven answer set**.

next

Excellent 👍

Here is the **FINAL PART – PART 5**, completing your **resume-driven, non-core, Amdocs-style interview answers**.

◆ PART 5

FUTURE, COMPANY-FIT & ENTERPRISE READINESS

1. Which project are you most proud of and why?

"I am most proud of the **Automatic Attendance System** because it combines machine learning, hardware constraints, and real-world usability. Deploying it on Raspberry Pi and achieving high accuracy taught me how theory works in practical environments."

2. If given 6 months, which project would you convert into a product?

"I would convert **DailyFit** into a product because it solves a real-world problem and has clear scope for scaling, monetization, and feature expansion."

3. How would you expand DailyFit in 6 months?

"I would add real-time availability updates, better search and filters, analytics for gym owners, secure payment handling, and scalable deployment."

4. How do your skills fit Amdocs' work culture?

"My skills align with Amdocs' focus on large-scale systems, structured development, and continuous learning. I am comfortable working on backend logic, problem solving, and collaborating in team-based environments."

5. How comfortable are you learning enterprise-level systems?

"I am comfortable learning enterprise systems. My projects required understanding complete flows, handling edge cases, and working with constraints, which prepares me well for enterprise-level applications."

6. How do you handle complex systems that you don't fully understand initially?

"I break the system into smaller modules, understand each part step by step, refer to documentation, and ask questions when needed."

7. How do you see yourself growing at Amdocs?

"I see myself growing as a reliable software engineer who understands both technical and business aspects of systems, and gradually taking ownership of larger modules."

8. Why should Amdocs invest in you?

"I have strong fundamentals, hands-on project experience, and a mindset of continuous learning. I am motivated to grow with the organization and contribute long-term."

9. Final question you can ask the interviewer

- "What technologies will freshers mostly work on in the first year?"
 - "How does Amdocs support learning and upskilling?"
 - "What defines success for a fresher at Amdocs?"
-

FINAL AMDOCS INTERVIEW TAKEAWAY

- Your interview will be **resume-driven**
 - Every feature you mention can be **deeply questioned**
 - Calm explanation > fancy terms
 - Ownership + learning mindset matters most
-
-

AMDOCS – COMPANY-SPECIFIC QUESTIONS & ANSWERS

1 Why do you want to join Amdocs?

“Amdocs works on large-scale telecom and enterprise systems that impact millions of users. I want to start my career in a company where I can work on real production systems, learn industry-level practices, and grow technically.”

2 What do you know about Amdocs?

“Amdocs is a global software company that provides solutions for telecom operators and enterprises, mainly in areas like billing, customer experience, and digital services.”

3 What makes Amdocs different from other IT companies?

“Amdocs focuses on **long-term enterprise solutions**, not short-term projects. The scale, complexity, and impact of its systems make it different.”

4 Why Amdocs for a fresher?

“Amdocs has structured training, strong mentorship, and exposure to large systems, which is very important for a fresher’s foundation.”

5 Why should Amdocs hire you?

“I have strong fundamentals, hands-on project experience, and a learning mindset. I take ownership of tasks and adapt quickly to new technologies.”

6 How do your skills align with Amdocs?

“My experience in backend logic, data handling, problem solving, and system thinking aligns well with Amdocs’ enterprise software development.”

7 What do you expect from Amdocs?

“I expect learning opportunities, mentorship, and exposure to real-world systems that help me grow as a software engineer.”

8 What do you know about Amdocs’ work domain?

“Amdocs primarily works in telecom and digital services, helping operators manage billing, customer experience, and network-related operations.”

9 Are you comfortable working on legacy systems?

“Yes. I understand enterprise systems often include legacy components, and I’m comfortable learning and maintaining them.”

10 Are you ready to learn domain knowledge (telecom)?

“Yes. I see domain learning as an opportunity to grow and add long-term value.”

1 1 What motivates you to join Amdocs over startups?

“Amdocs offers system stability, deep technical exposure, and structured learning, which is ideal for the early phase of my career.”

1 2 How do you handle pressure in enterprise projects?

“I break tasks into smaller parts, prioritize work, and communicate clearly with my team.”

1 3 What if you’re assigned a technology you don’t know?

“I will learn it quickly by studying documentation, practicing, and seeking guidance.”

1 4 Are you comfortable with shifts and global teams?

“Yes, I’m comfortable working in shifts and collaborating with global teams.”

1 5 Where do you see yourself at Amdocs in 3–5 years?

“I see myself as a skilled engineer contributing to critical modules and taking ownership of larger responsibilities.”

1 6 What values do you bring to Amdocs?

“Consistency, ownership, teamwork, and a strong learning attitude.”

1 7 What questions would you ask us?

- How does Amdocs support freshers’ learning?
- What technologies are most used for new hires?
- How is performance evaluated for freshers?